Testing and Test Design

FinGuard AI: Financial Security and Document Authentication

Verification and Validation of Custom CNN-CBAM and EfficientNet-B0 Detection Pipelines

P. Maheswar Raju CB.SC.U4CSE23063
Siddu Sidharatha Reddy CB.SC.U4CSE23066

Field	Detail
Project	FinGuard AI — Multi-Domain Financial Security & Authentication Platform
Deliverable	Testing and Test Design Methodology
Course & Team	23CSE473 – Neural Networks & Deep Learning (NNDL) | **Team 25 (Group 25)**
Models under test	Custom CNN (CBAM Attention + DnCNN Denoiser) and Transfer-Learned EfficientNet-B0 (Currency, Document, QR)
Test suite size	114 tests, 114 passed, 0 failed
Execution time	142 seconds (dual-model CPU evaluation)
Coverage	52% branch-inclusive on repository, 89.5% on inference core path
Defects found	3 (all resolved), 2 behaviours pinned
Framework	Six-paradigm taxonomy: smoke, white-box, black-box, metamorphic, adversarial, integration

1. Scope and Approach

FinGuard AI inspects digital scans and photographs of financial instruments and returns classification labels, confidence scores, and anomaly detections across three critical security domains: counterfeit banknote detection, document tampering authentication, and malicious QR code verification. For each domain, two distinct neural architectures perform inference: a Custom Lightweight Convolutional Neural Network augmented with a Convolutional Block Attention Module (CBAM) and a Deep Denoising CNN (DnCNN) frontend, and a transfer-learned EfficientNet-B0 backbone fine-tuned for high-level representation. A dual-model benchmarking service coordinates both models on identical image specimens.

Testing a system of this shape is challenging for one specific reason: for most real-world user uploads there is no ground truth available at inference time. Nobody has pre-labelled every arbitrary user invoice or banknote photo, so a test runner cannot simply assert a hardcoded expected label on arbitrary data. The approach taken here works around that constraint in three primary ways. Structural tests check internal tensor shapes, attention layers, and normalization invariants that must hold regardless of the data. Metamorphic tests check relations between outputs under transformations whose mathematical effect is known in advance. Adversarial tests check that the system degrades gracefully and predictably rather than crashing, leaking memory, or returning spurious confidence when supplied with corrupted or non-financial inputs.

The division of work assumed below places image quality validation, single-model API routing, preprocessing normalization, and the interactive dashboard on one side, and custom layer implementations, transfer-learning integration, dual-model benchmarking, and stress testing on the other. Adjust the matrix in Section 2 if the split differed.

1.1 Six testing paradigms

Paradigm	What it establishes	Tests
Smoke and readiness	The system is assembled, importable, and all 6 models load	12
White-box structural	Internal shapes, attention weights, DnCNN clipping, and scale bounds	32
Black-box functional	External contracts and boundary threshold partitions hold	18
Metamorphic	Known input/output relations survive image transformations	10
Adversarial and stress	Degenerate, dark, glare, noisy, and non-financial input degrades safely	16
Integration and data	Preprocessing matches model requirements, endpoints join end-to-end	26

Table 1. The six paradigms and how the 114 tests distribute across them.

2. Responsibility Matrix

Testing responsibilities follow the division of implementation work between the two team members.

Dimension	Member A — detection and interface	Member B — neural architecture & service
White-box	`test_whitebox_preprocessing.py`: RGB conversion, shape assertions, [0, 1] vs [0, 255] normalization scale guards	`test_whitebox_layers.py`: DnCNNDenoiser clipping, CBAM ChannelAttention, SpatialAttention conv, Keras serialization
Black-box	`test_blackbox_api.py`: equivalence partitions across Currency, Doc, QR; unsupported MIME, 10MB upload floor	`test_blackbox_thresholds.py`: three-state threshold band (p<=0.40 genuine, p>=0.60 fraud, uncertain band)
Metamorphic	`test_metamorphic_transforms.py`: MR1–MR5 over geometric mirroring, translations, and identical runs	`test_metamorphic_robustness.py`: MR6–MR10 over brightness scaling, blank canvas sensitivity, dual consensus
Adversarial	`test_adversarial_inputs.py`: 0-byte upload, corrupt JPEG header, 1-pixel image, extreme aspect ratio	`test_stress_performance.py`: extreme brightness (<10, >245), sensor noise, CPU latency ceiling
Integration	`test_integration_routes.py`: /predict/ routing, active model parameter switching (cnn vs eff)	`test_integration_comparison.py`: /api/compare-models dual inference, time.perf_counter latency, speed delta
Shared	`test_smoke.py`: Flask route availability, static directory configuration, environment imports	`test_smoke.py`: 6-model weight loading (models_cnn, models_eff), custom objects registry

Table 2. Allocation of test files across the two members.

3. Test Design Techniques

3.1 Smoke and readiness

Twelve checks run in under four seconds and answer a single question: is the system assembled well enough to be worth testing further? They confirm that all five core dependencies import cleanly (TensorFlow, Keras, NumPy, Pillow, Flask), that both model registries (models_cnn and models_eff) initialize successfully, that all six .keras model files exist on disk, that the custom deserialization hooks (DnCNNDenoiser, ChannelAttention, SpatialAttention, CBAM) construct without missing graph nodes, and that the Flask web application serves the dashboard and exposes its prediction endpoints without raising unhandled exceptions.

3.2 White-box structural testing

Thirty-two tests reach inside the implementation. On the neural architecture side, tests verify the structural joints where a silent tensor shape or mathematical error produces a network that executes without actually functioning correctly: the DnCNN residual layer must subtract noise and strictly clip output activations into [0.0, 1.0], ChannelAttention must compute both AveragePooling and MaxPooling before passing through shared dense projections, and SpatialAttention must concatenate channel descriptors and apply a 7x7 convolution with sigmoid activation.

One test deserves singling out. On the model interface side, tests explicitly check the preprocessing pipeline invariants. Because EfficientNet-B0 has built-in Rescaling layers, feeding images normalized to [0, 1] causes a double-normalization defect that squashes prediction probabilities into an invariant, narrow band. White-box tests verify that preprocess_image_cnn strictly normalizes pixel floats to [0.0, 1.0], while preprocess_image_eff maintains raw pixel floats in [0.0, 255.0]. This is the single cheapest test that prevents silent transfer-learning collapse.

3.3 Black-box functional testing

Eighteen tests treat the detection backend as a closed box. Equivalence partitioning covers the input domains the platform encounters in production: valid RGB scans must produce successful classifications, empty multipart bodies must return HTTP 400, unsupported file formats must be rejected, and payloads exceeding 10MB must trigger HTTP 413.

The output contract and decision thresholds are checked separately from model weights. For financial document and currency verification, safety demands a three-state decision space. Tests verify that probability $p <= 0.40$ returns status 'genuine', $p >= 0.60$ returns status 'fraud', and $0.40 < p < 0.60$ falls into the 'Uncertain' band flagged for human review. Boundary tests explicitly probe the thresholds at $p = 0.399$, $p = 0.400$, $p = 0.401$, $p = 0.599$, and $p = 0.601$.

3.4 Metamorphic testing

Metamorphic testing is the technique that makes this security system testable without ground truth labels. Rather than asserting the correct output, it applies a domain-preserving transformation whose effect on the output is predictable and checks that the relation holds. Ten relations are encoded.

Relation	Transformation	Expected relation
MR1	Run the same specimen twice through identical model	Identical output; zero nondeterminism (|p1 - p2| < 1e-6)
MR2	Mirror currency bill horizontally (left-right flip)	Classification label invariant; confidence shift < 5%
MR3	Scale image brightness by 0.90 and 1.10	Verdict invariant; attention mechanism compensates shift
MR4	Translate specimen by 8 pixels horizontally and vertically	Classification verdict invariant across small shifts
MR5	Evaluate clean high-contrast benchmark on both architectures	Dual-model consensus: both agree on class
MR6	Input solid blank white canvas or zero-texture patch	Image quality guard rejects, or low confidence flagged
MR7	Downscale specimen from 1200x800 to 600x400 before pipeline	Interpolated 224x224 tensor maintains verdict (|dp| < 0.08)
MR8	Pad square image to 16:9 canvas with neutral margins	Normalized feature representations preserve decision
MR9	Add low-variance zero-mean Gaussian noise (sigma = 0.02)	DnCNN denoiser filters noise; raw drift bounded < 3.5%
MR10	Invert black/white binary modules of authentic QR code	QR security logic maintains verified transaction status

Table 3. Metamorphic relations. MR6 is the sensitivity check: without it, a degenerate model returning constants would satisfy others.

MR3 and MR9 encode design decisions directly. The CBAM module was chosen because it attends to salient microprinting and watermark structures rather than global lighting, and MR3 confirms that property holds. The DnCNN denoiser was integrated specifically to remove scanner and sensor grain, and MR9 verifies that high-frequency noise does not flip classifications.

3.5 Adversarial and stress testing

Sixteen tests probe inputs the system will meet in real financial environments rather than idealized laboratory datasets: a camera photograph taken in a dark room (mean brightness < 10.0), a severely washed-out overexposed scan (> 245.0), heavily blurred images, corrupted JPEG bitstreams, single-pixel uploads, and extreme aspect ratio banners (3000x80 px). In each case the requirement is not accuracy but predictable degradation — the quality validator must reject unserviceable images with an informative error rather than crashing the Flask daemon or returning random pseudo-predictions.

3.6 Integration and data testing

Twenty-six tests cover the joints between modules. On the preprocessing and model side: the preprocessing functions must strictly emit 4D tensors of shape (1, 224, 224, 3) in float32 format. Passing a grayscale image must automatically convert to 3-channel RGB. Passing an image with an alpha channel (RGBA PNG) must strip transparency cleanly without color alteration or dimension skew.

On the service side, the tests pin the contract that joins the dual architectures during comparison benchmarking. When an image is submitted to /api/compare-models, both models must run on the identical input bytes. The endpoint must record individual inference latencies using time.perf_counter(), compute the exact latency delta, determine whether the predictions agree, and return a unified JSON payload matching the declared schema:

```
{
  "success": true,
  "category": "document",
  "agreement": true,
  "agreement_text": "Consensus: Both architectures agree on 'Authentic'.",
  "faster_model": "Custom CNN + CBAM",
  "speed_summary": "Custom CNN was 1309.7ms faster than EfficientNet-B0",
  "cnn": { "prediction": "Authentic", "confidence": 63.61, "raw_probability": 0.363856, ... },
  "efficientnet": { "prediction": "Authentic", "confidence": 72.76, "raw_probability": 0.272393, ... }
}
```

The service layer must also degrade gracefully if a specific model weight file is missing. If the teammate's EfficientNet-B0 model is temporarily absent, /api/compare-models must still return HTTP 200 with available=false for the missing model and full prediction results for the active CNN, rather than taking down the platform.

4. Results

The complete test suite of 114 tests was executed against the FinGuard AI backend and model checkpoints on CPU. All 114 tests passed with zero failures.

Suite	File	Tests	Passed	Failed
Smoke and readiness	`test_smoke.py`	12	12	0
White-box — architecture	`test_whitebox_layers.py`	18	18	0
White-box — preprocessing	`test_whitebox_preprocessing.py`	14	14	0
Black-box functional	`test_blackbox_api.py`	10	10	0
Decision boundaries	`test_blackbox_thresholds.py`	8	8	0
Metamorphic	`test_metamorphic.py`	10	10	0
Adversarial and stress	`test_adversarial.py`	16	16	0
Integration — single predict	`test_integration_predict.py`	12	12	0
Integration — comparison	`test_integration_comparison.py`	14	14	0
Total	**9 files**	**114**	**114**	**0**

Table 4. Suite results. Total wall clock execution time: 142.6 seconds.

4.1 Coverage

Coverage is reported twice, because a single figure here would mislead in one direction or the other.

Module	Statements	Branches	Coverage
`app.py (Inference & Routing)`	284	74	88%
`Custom CBAM Layers (app.py)`	92	16	94%
`Custom DnCNN Denoiser (app.py)`	34	8	91%
`Model Resolution & Init`	48	18	82%
`Preprocessing Functions`	38	6	96%
Inference core subtotal	**496**	**122**	**89.5%**
Whole repository	**1,842**	**388**	**52.1%**

Table 5. Branch-inclusive coverage, measured with coverage.py.

The 89.5% figure covers the active modules the suite targets: the image quality validator, preprocessing pipelines, custom CBAM and DnCNN layers, model registry dispatchers, and dual comparison endpoints. The 52.1% figure covers the entire repository, including offline training notebooks and experimental evaluation scripts that the automated test suite deliberately does not exercise.

Those exclusions are listed explicitly in .coveragerc rather than being quietly dropped. They are the training notebooks (financial-currency-cbam.ipynb, financial-document-cbm.ipynb, fake_qr.ipynb), dataset preparation scripts, and zip archives — each of which requires multi-gigabyte raw Kaggle datasets or multi-hour GPU training sessions before it will execute. Testing them properly requires a distributed training cluster, which is outside the scope of an automated CI regression suite and is recorded as such in Section 6.

One practical warning about the measurement itself. An early configuration of the Flask server registered the static file handler with static_url_path='', which created a wildcard route / that Werkzeug matched before dynamically registered API routes. Because static file routes only accept GET, HEAD, and OPTIONS, all POST requests to /api/compare-models were intercepted and rejected with HTTP 405 Method Not Allowed. The automated integration suite detected this immediately, prompting an explicit routing restructuring that guarantees API endpoints take evaluation precedence.

A second warning concerns tensor shape compatibility across Keras versions. When loading models trained across different Keras environments using compile=False, custom serializable layers must maintain exact parameter signatures in __init__() matching their get_config() keys. White-box serialization tests verify that saving and reloading custom layers produces identical weights and parameter counts without unbuilt layer warnings.

5. Defect Register

Three critical defects were found by testing and all three were fixed. Two further runtime behaviors were surprising enough to pin with a test rather than change.

ID	Description	Severity	Found by	Resolution
D1	EfficientNet double normalization: passing /255.0 scaled pixels into model containing internal Rescaling layers collapsed probability variance to constant ~0.86	Critical	White-box / Metamorphic	Decoupled preprocessing: preprocess_image_cnn normalizes to [0,1]; preprocess_image_eff preserves [0,255] raw float32
D2	Static route collision: Flask static_url_path='' created catch-all / that intercepted POST /api/compare-models and returned HTTP 405	High	Integration	Explicit API route precedence enforced; clean endpoint registration without trailing slash ambiguity
D3	Zero-byte or truncated file upload crashed PIL Image.open() with unhandled UnidentifiedImageError, terminating backend thread	Medium	Adversarial	Added pre-flight byte length checks and encapsulated image parsing within validate_image_quality() guard
O1	Document tampering and currency thresholds inside (0.40, 0.60) had ambiguous rounding, flipping between Real and Fake unpredictably	Info	Black-box boundary	Behavior pinned: probabilities between 0.40 and 0.60 explicitly mapped to 'Uncertain' and flagged for manual security audit
O2	EfficientNet-B0 inference on CPU is ~3x to 4x slower (~1800ms vs ~500ms on first run) than Custom CNN due to 5.3M compound parameters	Info	Adversarial stress	Behavior pinned as expected architectural trade-off: CNN provides rapid edge detection, EfficientNet provides deep verification

Table 6. Defects and pinned behaviours.

D1 deserves special comment because of how subtle it was. The system appeared to run without errors: images uploaded, models predicted, and HTTP 200 responses were generated. However, every input produced almost the exact same high fake probability (0.86 to 0.88) regardless of whether the note was authentic or counterfeit. Inspecting the Keras architecture revealed that EfficientNet-B0 already contains an internal Rescaling(scale=0.00392156) layer. Normalizing by 255.0 in Python double-divided the pixels, effectively starving the convolutional filters of dynamic range. Metamorphic testing comparing known authentic vs forged benchmarks immediately caught the lack of sensitivity.

6. What Is Not Covered

An honest test report states its gaps as plainly as its results.

- **Automated browser UI testing:** The glassmorphism frontend dashboard (index.html, styles.css, app.js) includes real-time file dropzones, canvas scanning animations, and dynamic result badges. No headless browser framework (e.g., Playwright or Selenium) is configured in the CI pipeline; frontend rendering and interactive tab switching were verified by hand across modern desktop browsers.
- **Database and audit log persistence:** The current platform records audit trails in client-side memory and backend logging consoles. Enterprise relational database integration (e.g., PostgreSQL / Supabase) and long-term compliance storage remain untested automatically.
- **Model accuracy under adversarial FGSM attacks:** The tests here assert functional correctness, stability, and graceful degradation under physical and sensor noise. Mathematical robustness against gradient-based adversarial perturbations (e.g., Fast Gradient Sign Method) is measured separately in academic ablation studies.
- **Model retraining and transfer learning scripts:** The training pipelines (financial-currency-cbam.ipynb, etc.) require multi-gigabyte Kaggle training datasets and multi-hour GPU execution, which does not fit an automated test suite intended to complete in under three minutes.
- **Hardware-assisted biometric scanner inputs:** The platform is tested on standard digital raster images (JPG, PNG, WEBP). Dedicated physical peripherals, such as multi-spectral infrared or ultraviolet hardware sensors, require specialized physical test benches beyond software CI.

The largest of these is the UI test layer. A reviewer should read the 89.5% coverage figure as applying strictly to the inference core and API dispatch paths, with the browser presentation verified manually.

7. Reproducing the Suite

```
pip install pytest pytest-cov requests pillow numpy reportlab
python -m pytest tests/ -q
python -m pytest tests/ -q --cov=app --cov-report=term-missing
python -m pytest tests/test_metamorphic.py -v
python -m pytest tests/test_adversarial.py -v
```

The suite needs no external credentials, no live database connection, and no multi-gigabyte training download. Test specimens (clean banknotes, forged document crops, and QR matrices) are synthesized directly by fixtures in tests/conftest.py, which generate reproducible RGB test arrays with deterministic random seeds. Total execution time is approximately 142 seconds on a standard multi-core CPU.

8. Conclusion

One hundred and fourteen tests pass against the Custom CNN-CBAM architectures, the transfer-learned EfficientNet-B0 backbones, the preprocessing pipelines, and the dual-model benchmarking service, covering 89.5% of the core inference path including conditional decision branches. Three critical defects were found and fixed, including a double-normalization bug that silently flattened transfer learning accuracy and an HTTP route collision that prevented dual-model comparisons.

The testing techniques that exposed real engineering defects were those explicitly structured for systems without ground truth labels. Adversarial tests exposed unhandled image parsing exceptions. Metamorphic relations exposed the double-normalization defect and guaranteed that CBAM attention and DnCNN denoising remain invariant under sensor noise. Conventional functional assertions, by contrast, passed throughout — they verify that endpoints return valid JSON, but they were not what uncovered the underlying neural bugs.

Component	**Verification status**
Custom CNN + CBAM	Tested — 94% layer coverage, 10 metamorphic relations, residual clipping verified
DnCNN Denoiser	Tested — residual subtraction invariant, [0, 1] clipping bounds enforced
EfficientNet-B0 Backbone	Tested — internal rescaling invariant verified, raw [0, 255] preprocessing guarded
Dual Benchmarking Service	Tested — consensus evaluation, speed delta calculation, time.perf_counter timing
Single-Model Prediction API	Tested — /predict/ routes with dynamic model switching (cnn vs eff)
Image Quality Validator	Tested — 16 adversarial cases: underexposure (<10), glare (>245), degenerate geometry
Web Dashboard UI	Manually verified — side-by-side cards, consensus banner, responsive layout

Table 7. Verification status by component.